Comparison of selected metaheuristic algorithms

Systems and Decision Methods semester project at Wrocław University of Science and Technology

Selected metaheuristic algorithms:

- Ant Colony Optimization
- Simulated Annealing
- Bees Algorithm

Authors:

- Przemysław Mazurowski
- Bartłomiej Niedbała
- Szymon Majerczak

Submission date: 2026/05/31

---

1. The Travelling Salesman Problem (TSP) characteristics

The travelling salesman problem is an NP-hard problem in the theory of computational complexity. It does ask the following question: *Given a list of cities and the distances between each pair of cities, what is the shortest possible route that visits each city exactly once and returns to the origin city?*

1.1. Definition

Formally TSP is an optimization problem that consists of finding the minimum Hamiltonian cycle in a complete weighted graph.

Two types of TSP problem can be pointed:

- STSP - Symmetric Travelling Salesman Problem
- ATSP - Asymmetric Travelling Salesman Problem

In the Symmetric Travelling Salesman Problem (STSP), the weight of edge (i, j) never differs from weight of (j, i) edge for every pair of nodes. In the Asymmetric Travelling Salesman Problem (ATSP), however, these distances may vary. In this report only STSP is considered and any reference to TSP refers to STSP.

1.2. Problem complexity

A defining characteristic of the TSP is the fact that it is the NP-hard problem - any proposed solution, specific sequence of nodes, can be efficiently verified in polynomial time. Nonetheless, no algorithm has been discovered that can solve the TSP in polynomial time. Furthermore, TSP holds a significant place in computational complexity - every single NP problem can be reduced to an instance of TSP in polynomial time.

The TSP presents a massive computational challenge because its complexity grows factorially. For a TSP with n cities, the total number of unique routes that can be evaluated is given by the formula:

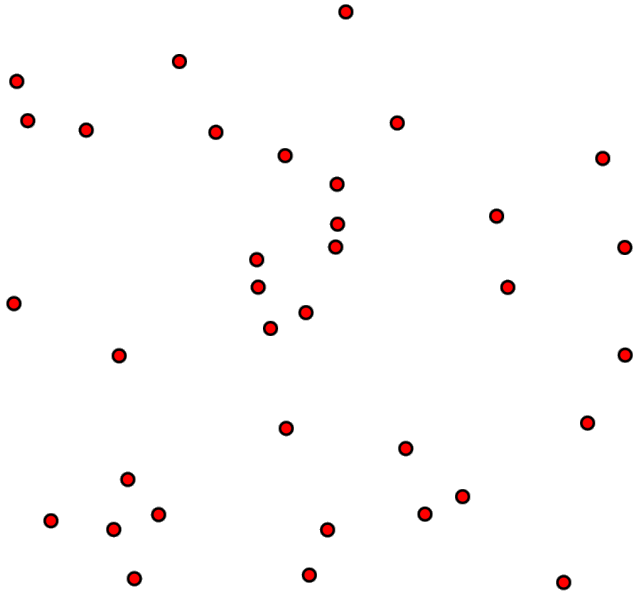
$$\frac{(n-1)!}{2}$$

This combinatorial explosion means that even for a small instance of just 20 cities, the scale of available combinations expands rapidly:

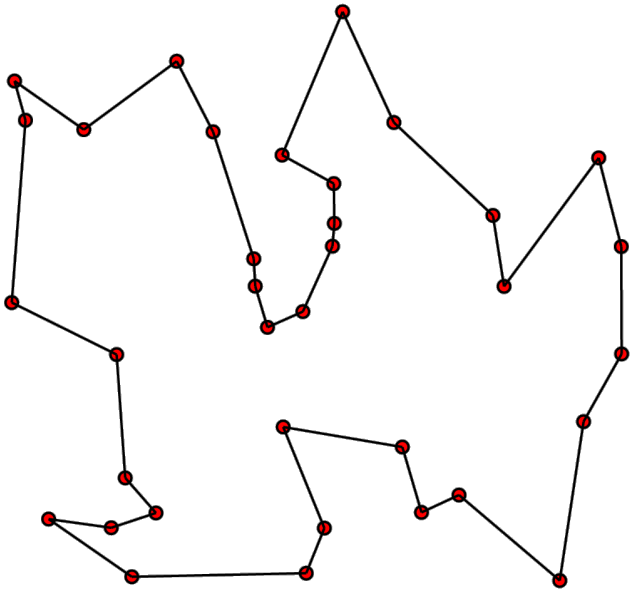
$$\frac{19!}{2} \approx 6 \times 10^{16}$$

As a result, finding an exact optimal solution through brute-force computation becomes practically impossible as the number of locations increases. Way of finding near-optimal solutions to the TSP is applying metaheuristic algorithms.

1.3 Visualisation of the TSP instance and optimal solution



Unsolved TSP instance.



Solution of the TSP instance.

Sources:

- https://en.wikipedia.org/wiki/Travelling_salesman_problem
- https://pl.wikipedia.org/wiki/Problem_komiwoja%C5%BCera
- https://en.wikipedia.org/wiki/NP-hardness
- https://en.wikipedia.org/wiki/Travelling_salesman_problem#/media/File:GLPK_solution_of_a_travelling_salesman_problem.svg
- https://en.wikipedia.org/wiki/Travelling_salesman_problem#/media/File:Illustration_of_an_unsolved_travelling_salesman_problem.svg

---

2. Solution representation

The TSP solution for n vertices can be represented as a permutation π of the vertex set, where v_i denotes the i-th vertex in the sequence:

$$\pi = (v_1, v_2, \ldots, v_n)$$

The weights of edges - distances - are retrieved from a distance matrix D, where each element $d_{i,j}$ represents the cost of traveling from vertex i to vertex j:

$$D = [d_{i,j}]_{n \times n}$$

The objective of the optimization is to find an optimal tour π^* that minimizes the cost function $F(\pi)$. This function calculates the sum of distances between consecutive vertices and includes the distance from the last vertex back to the first one to close the cycle:

$$F(\pi) = \sum_{i=1}^{n-1} d_{v_i, v_{i+1}} + d_{v_n, v_1}$$

$$\pi^* = \arg\min_{\pi \in \Pi} F(\pi)$$

where Π denotes the set of all possible permutations - valid tours.

---

3. Overwiew of selected alghorithms

3.1 Ant Colony Optimization

Ant System is a general-purpose heuristic algorithm which can be used to solve various optimization problems. It is a population-based approach. It was proposed by Marco Dorigo, Vittorio Maniezzo and Alberto Colorni in 1996 in the paper *Ant System: Optimization by a Colony of Cooperating Agents*. In this system, search activities are distributed over so-called *ants* – agents with basic capabilities based on the behaviour of real ants. In fact, those agents possess capabilities beyond those of natural ants, like memorizing their paths.

Ant System is inspired by ants establishing the shortest routes from their colonies to feeding resources and back. The medium used by ants to set up those paths consists of pheromone trails. Each moving ant lays pheromone in different quantities. Ants tend to choose paths with more pheromone left while reinforcing the chosen path with their own pheromones. The shortest paths are eventually established, since shorter paths are covered with higher frequency than the longer ones.

The implementation is based on ant-cycle algorithm introduced in the original paper. However it does differ from original version – the stopping criterion relies on the maximum number of cost function evaluations instead of a fixed number of cycles and detecting stagnation behaviour is based on exceeding limit of cycles without improvement rather than on all ants traversing the same path. While the implementation remains semantically equivalent to the original algorithm, certain traditional loops have been replaced with optimized, vectorized operations.

The hyperparameters of the ant-cycle optimization algorithm:

- m - The total number of ants in the colony.
- c - The initial pheromone trail intensity assigned to each edge.
- p - The pheromone retention coefficient, where $(1 - p)$ represents the evaporation rate.
- q - A constant used to scale the amount of pheromone laid by ants on traversed edges.
- α - A parameter controlling the influence of the pheromone trail on the ant's routing decisions.
- β - A parameter controlling the influence of actual distance on the ant's routing decisions.

Additionalny, in the implemention examined in this report, *cost function evaluation limit* and *cycles without best cost improvement limit* are to be set.

Hyperparamters for each instance optimization where found using *Optuna*.

3.2 Simulated Annealing

Simulated Annealing (SA) is a probabilistic, single-state metaheuristic algorithm used to locate global optima within large and complex search spaces. It is particularly effective for discrete optimization problems like the TSP, where traditional gradient-based approaches fail due to non-differentiable or highly rugged objective landscapes.

The algorithm is inspired by metallurgy, where a material is heated to a high temperature and then cooled slowly and systematically. Heating increases the kinetic energy of the atoms, allowing them to break out of their initial, flawed configurations. Slow cooling then permits them to settle into highly ordered, low-energy crystalline states. In the context of mathematical optimization, this physical metaphor maps directly to the algorithm's components:

- **State / Solution:** A specific permutation of cities representing a valid TSP tour (π).
- **Energy:** The value of the objective function (F(π)) evaluated at a given state. Because the objective is to minimize total distance, lower energy corresponds to a better solution.
- **Temperature (T):** A global control parameter that dictates the degree of randomness in the search space exploration.

The Metropolic Acceptance criterion

The defining feature of Simulated Annealing is its ability to escape local optima by accepting worse solutions based on a probabilistic threshold, preventing the algorithm from stalling like a purely greedy hill-climber.

When moving from a current tour with energy E_{current} to a neighboring tour with energy E_{new}, the change in energy is computed as: $\Delta E = E_{\text{new}} - E_{\text{current}}$

Improving Moves ($\Delta E < 0$): If the neighbor solution yields a shorter path, it is always accepted.

Degrading Moves ($\Delta E \geq 0$): If the neighbor solution is worse or equal, it is accepted with a probability P determined by the Metropolis criterion:

$$P = \exp\left(-\frac{\Delta E}{T}\right)$$

Adaptive Initial Temperature Estimation

Simulated Annealing exhibits high sensitivity to the choice of the initial temperature (T_0). Setting T_0 too high wastes an excessive number of function evaluations on a purely random walk, while setting it too low causes the algorithm to prematurely collapse into a greedy local search, getting trapped in the nearest local minimum.

To eliminate arbitrary static configurations, this implementation dynamically estimates T0 based on empirical performance testing. Before starting the optimization loop, the algorithm performs preliminary

sampling:

- It begins with a random solution, finds a neighbor, transitions to it, and repeats this chain for 1000 samples.
- It calculates the mean value of the energy increases (ΔE) for degrading moves.
- It computes T_0 to guarantee that the initial acceptance probability of worse solutions is approximately 80% ($\chi_0 \approx 0.80$).

This relationship is derived by rearranging the Metropolis criterion:

$$P(acceptance) = \exp\left(-\frac{\overline{\Delta E}}{T_0}\right) \approx \chi_0 \Rightarrow T_0 = -\frac{\overline{\Delta E}}{\ln \chi_0}$$

Geometric Cooling Schedule

To lower the temperature at each iteration k, a **Geometric Decrease** profile is utilized:

$$T_{k+1} = \alpha T_k$$

Neighbor Operator

Two primary transition operators were evaluated during development phase:

- **Swap operator:** Interchanges the position of two randomly chosen cities in the permutation vector.
- **2-opt Operator:** Selects two edges from a route, removes them, reverses the sub-route between them and reconnects the edges to form a new valid cycle.

Because the 2-opt operator proved significantly more efficient at discovering lower-energy configurations during preliminary testing, the swap operator was discarded from the final implementation.

Hyperparameters:

- T_0 - initial temperature. Based on a Github repository we decided to approxiate the initial temperature with adaptive initial temperature estimation.
- α - cooling rate. Needs to strictly be in range (0, 1). Typically is set to be in range (0.95, 0.9999999999) depending on the problem size.
- Function evaluations was set to 3 000 000.

The hyperparameters will be optimized using *Optuna*.

3.3 Bees Algorithm

The Bees Algorithm (BA) is a population-based metaheuristic inspired by the food foraging behaviour of honey bees. It was introduced by D. T. Pham, A. Ghanbarzadeh, E. Koc, S. Otri, S. Rahim and M. Zaidi in the paper *The Bees Algorithm - A Novel Tool for Complex Optimisation Problems*. The algorithm combines neighbourhood search around promising solutions with random exploration of new areas of the search space.

In the natural metaphor, scout bees search for food sources. The best discovered flower patches attract more recruited bees, while weaker patches receive fewer bees. At the same time, some scouts continue exploring randomly, which helps the colony avoid premature convergence. In the TSP context, a flower patch corresponds to a candidate tour, and the nectar amount is represented by the inverse quality of the tour cost. Since the objective is to minimize total route distance, lower cost means a better site.

The implementation examined in this report follows this population-based structure. First, an initial population of valid TSP tours is generated. In order to avoid spending a large part of the evaluation budget on very poor random permutations, part of the initial population is constructed with the nearest-neighbour heuristic. The algorithm evaluates nearest-neighbour tours started from different cities and keeps the best constructed tours as initial promising sites. The remaining sites are generated randomly by scout bees. Every tour is evaluated using the common `Problem.evaluate()` interface, which also updates the global function evaluation counter.

In every cycle the population is sorted by cost, the best sites are selected, and additional neighbourhood search is performed around them. Elite sites receive more recruited bees than the remaining selected sites. When an improving neighbour is found, the local search continues from that improved tour, which makes the recruited bees perform a short greedy intensification around a promising site. The rest of the next population is completed by randomly generated scout bees.

The stopping criterion is the maximum number of cost function evaluations. Therefore, the algorithm does not stop after a fixed number of cycles. Instead, before every call to the objective function, the implementation checks whether the allowed number of evaluations has already been exhausted. This includes the nearest-neighbour initialization phase, so the constructive starts do not bypass the common evaluation budget. This makes the Bees Algorithm directly comparable with Ant Colony Optimization and Simulated Annealing implementations used in this project.

The main components of the implementation are:

- **Site / Solution:** A permutation of all cities representing a valid TSP tour.
- **Fitness / Cost:** The total route distance calculated by $F(\pi)$. Lower cost corresponds to a better food source.
- **Selected sites:** The best solutions chosen from the current population for neighbourhood search.
- **Elite sites:** The highest quality selected sites, explored with the largest number of recruited bees.
- **Scout bees:** Randomly generated solutions used to preserve exploration.
- **Constructive initial sites:** Nearest-neighbour tours used to start the population from more promising regions of the search space.

Neighbourhood operator

Two neighbourhood operators are supported by the implementation:

- **Swap operator:** Interchanges the positions of two randomly selected cities in the permutation.
- **2-opt operator:** Selects two positions in the tour and reverses the sub-route between them.

The 2-opt operator was selected as the default operator because it is better suited to TSP instances. Reversing a segment of the route can remove crossing edges and often produces much stronger improvements than a simple swap. In this project, `TSPProblem.get_neighbour()` provides a stochastic 2-opt move, while `TSPProblem.get_neighbour_swap()` keeps the swap-based operator for

comparison. The Bees Algorithm uses these shared operators without modifying the common TSP problem implementation.

Hyperparameters

The hyperparameters of the Bees Algorithm used in the implementation are:

- n - The total number of bees in the population (`n_bees`).
- n_{elite} - The number of best selected sites treated as elite sites (`n_elite`).
- n_{best} - The total number of selected sites used for neighbourhood search (`n_best`).
- r_{elite} - The number of recruited bees assigned to every elite site (`elite_neigh`).
- r_{best} - The number of recruited bees assigned to every non-elite selected site (`best_neigh`).
- d - The number of neighbourhood moves applied when creating a neighbouring solution (`neighbourhood_depth`).
- O - The selected neighbourhood operator, either 2-opt or swap (`neighbourhood_type`).
- g - The number of constructive nearest-neighbour tours retained in the initial population (`greedy_initial_sites`).
- s - The number of start cities tested during constructive initialization (`greedy_start_candidates`). In the performed experiments, all start cities were tested.
- k - The candidate list size used by the nearest-neighbour construction (`greedy_candidate_list_size`).

For the performed experiments, the following configuration was used:

- $n = 12$
- $n_{elite} = 2$
- $n_{best} = 4$
- $r_{elite} = 900$
- $r_{best} = 220$
- $d = 1$
- $O = \text{2-opt}$
- $g = 4$
- $s =$ all cities in the instance
- $k = 1$
- cost function evaluation limit: `50000`

---

4. Experiments

4.1 Selected instances

The experiments were conducted on three TSP instances of varying sizes. The tsplib95 library was utilized to obtain the problem instance files along with their optimal solutions, which were used to verify the effectiveness of the metaheuristics.

The three selected problems were:

- att48 (48 cities)
- eil101 (101 cities)

- and a280 (280 cities)

To evaluate the algorithms, tests were performed by executing them 10 times on each of the selected instances. Furthermore, to ensure an accurate comparison of execution times, all tests were conducted on the same computer.

4.2 Relative Percentage Deviation (RPD) Metric

Alongside values such as execution time and minimum costs, the Relative Percentage Deviation (RPD) metric was utilized in the presentation of the experimental results. This metric enables a more effective comparison of experimental results across different problem instances

$$\mathrm{RPD} = \frac{F_{\mathrm{best}} - F_{\mathrm{opt}}}{F_{\mathrm{opt}}} \cdot 100\%$$

Where F_{best} is the best cost found by the algorithm and F_{opt} is the known optimal cost for the given instance.

4.3 Stopping Criterions

The selected algorithms differ significantly from each other, making it impossible to apply a single, universal stopping criterion.

Ant System - the maximum number of cycles without improvement was set to 75, with a limit of 16 000 allowed cost function evaluations.

Simulated Annealing - the maximum number of iterations without improvement was disabled, we wanted to see how much can this algorithm improve solution, and later analyse the plots for stagnation.

Bees Algorithm - the maximum number of iterations like in the Simulated Annealing.

---

5. Obtained Comparative results

5.1 Results for att48 instance

Optimal cost: 10628.0

Algorithm	Best result	Best RPD	Avg result	Avg RPD	Avg exec time	Avg RPD std dev
`Ant System`	10965.0	3.17%	11225.8	5.62%	3.06	1.36%
`Simulated Annealing`	10628.0	0.00%	10755.3	1.20%	5.31	0.85%
`Bees Algorithm`	10711.0	0.78%	10857.0	2.15%	2.57	1.15%

5.2 Results for eil101 instance

Optimal cost: 629.0

Algorithm	Best result	Best RPD	Avg result	Avg RPD	Avg exec time	Avg RPD std dev
Ant System	671.0	6.68%	684.2	8.78%	12.61	1.25%
Simulated Annealing	639.0	1.59%	655.4	4.20%	7.78	1.35%
Bees Algorithm	647.0	2.86%	654.8	4.10%	3.14	0.77%

5.3 Results for a280 instance

Optimal cost: 2579.0

Algorithm	Best result	Best RPD	Avg result	Avg RPD	Avg exec time	Avg RPD std dev
Ant System	2964.0	14.93%	2991.2	15.98%	55.51	0.65 %
Simulated Annealing	2794.0	8.34%	2928.4	13.55%	8.55	2.95%
Bees Algorithm	2719.0	5.43%	2770.0	7.41%	3.39	0.76%

6. Individual Results and Conclusions

6.1 Ant system

The implemented Ant System, being the simplest variant of this algorithm, does not find optimal solutions. For the smaller tested instances, its results can be considered satisfactory. However, for the a280 instance, the best solution was noticeably worse—the lowest RPD reached as high as 15.98%.

The Ant System is stable - the best solutions it finds has a low standard deviation of the RPD. The execution time of the algorithm increases noticeably with the size of the optimized instance.

The algorithm finds solutions close to its final result in the first few to a dozen cycles. Therefore, the execution could be terminated much earlier by modifying the stopping criterion and accepting a near-final solution. Conversely, the algorithm could also be allowed to run longer. The standard deviation of the route costs among individual agents remains positive and varied over cycles, which indicates that the ants have not yet converged to a single path.

The following charts present the results for a single optimization run of the eil101 problem.

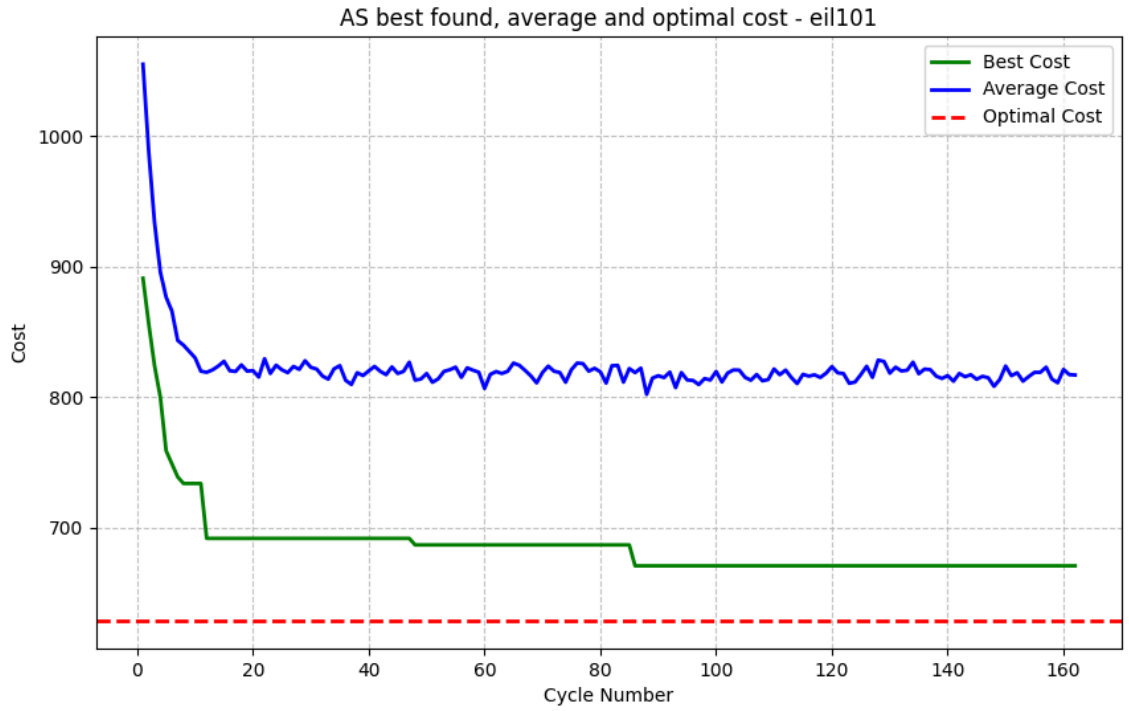


Figure 1: Optimization route cost for Ant System on eil101 instance.

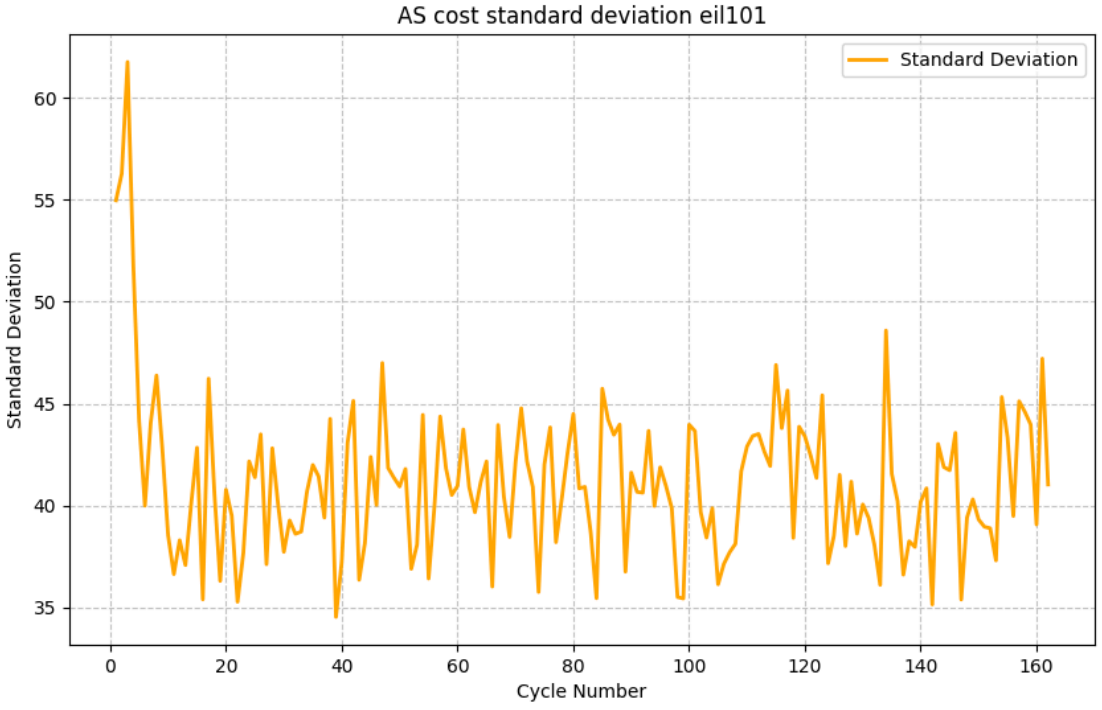


Figure 2: Standard deviation of agents route costs for Ant System on eil101 instance.

6.2 Simulated annealing

The implemented Simulated Annealing (SA) algorithm, enhanced by a 2-opt operator and adaptive initial temperature estimation, demonstrates high efficiency in solving small to medium-sized instances of the TSP. A key highlight of the algorithm's performance was observed in the smallest instance (att48), where SA was the only tested metaheuristic to successfully discover the global optimum, achieving a Relative

Percentage Deviation (RPD) of 0.00%. While the solution quality naturally decreases as the complexity of the search space grows, the final configurations remain highly satisfactory.

Experiments clearly indicate that despite its simplicity and single-state nature—relying on the trajectory of a single solution rather than evolving a population—Simulated Annealing can effectively compete with more complex metaheuristics. A critical factor in this capability was discarding the basic Swap operator in favor of the 2-opt move. The 2-opt operator generates a significantly less rugged fitness landscape, making it easier to systematically eliminate crossing edges. This approach is exceptionally well-suited for scenarios where the objective function is computationally cheap, as SA requires a high number of iterations to systematically cool the system and reach convergence.

The integration of Adaptive Initial Temperature Estimation paired with a finely tuned Geometric Cooling Rate (α) was a decisive factor in stabilizing the algorithm and maximizing its solution quality across varying instance sizes. Based on the low standard deviation (❤️%) among all experiments, we can say that SA is stable, but the least stable in comparison to the other algorithms.

Potential for Early Termination in Large Instances (`a280`): An analysis of the convergence trajectory for the `a280` instance reveals that the vast majority of the cost reduction occurs rapidly during the early and middle stages of the cooling cycle. Once the algorithm enters the lower temperature zones, the optimization curve flattens significantly, spending a considerable portion of the evaluation budget on minor, marginal refinements. This implies that for large-scale instances, the execution could be terminated much earlier—either by implementing much faster stagnation threshold.

The following charts present the results for the Simulated Annealing algorithm on 3 best runs for each problem.

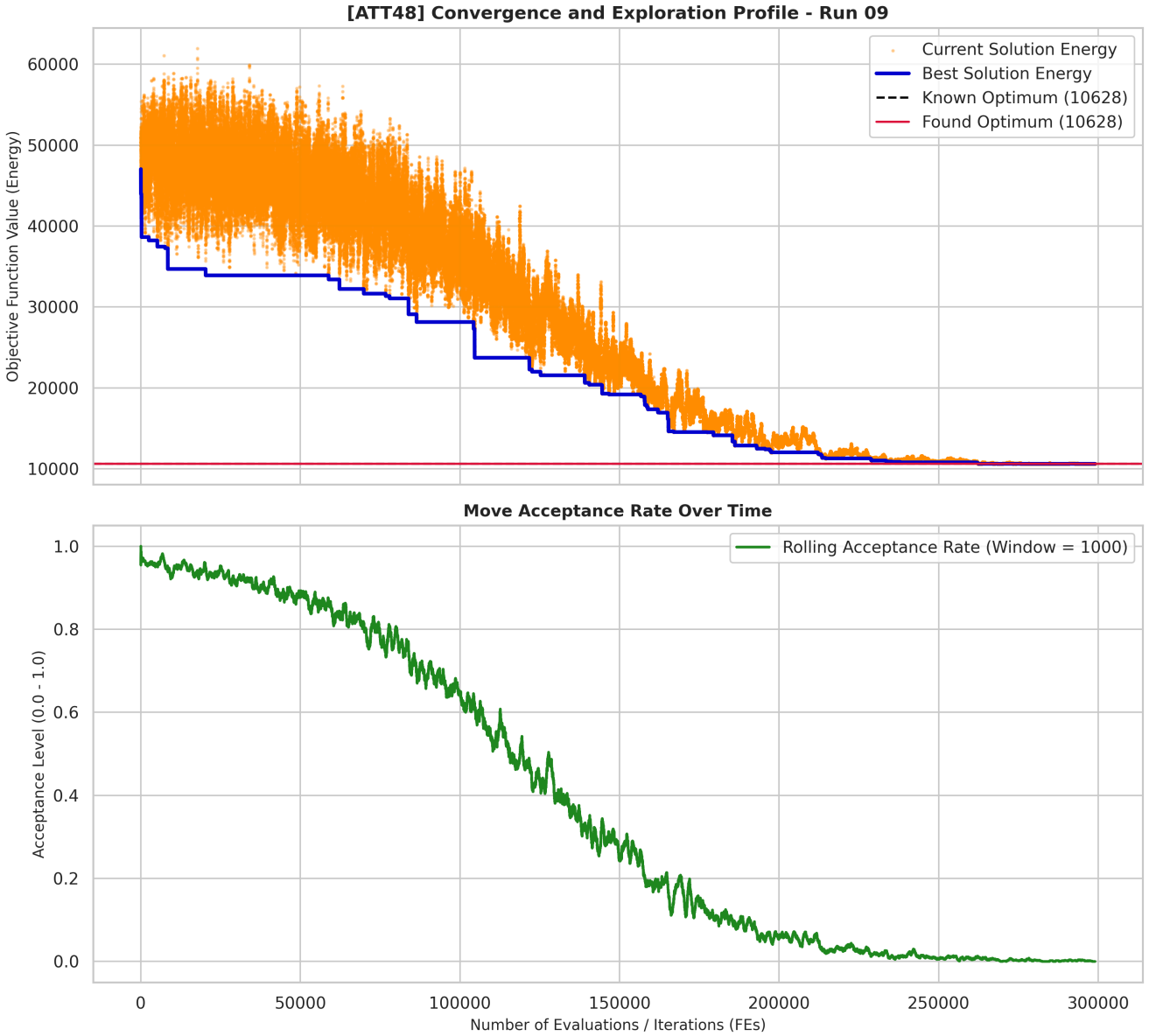


Figure 1: Optimization route cost and acceptance rate for SA on att48 instance.

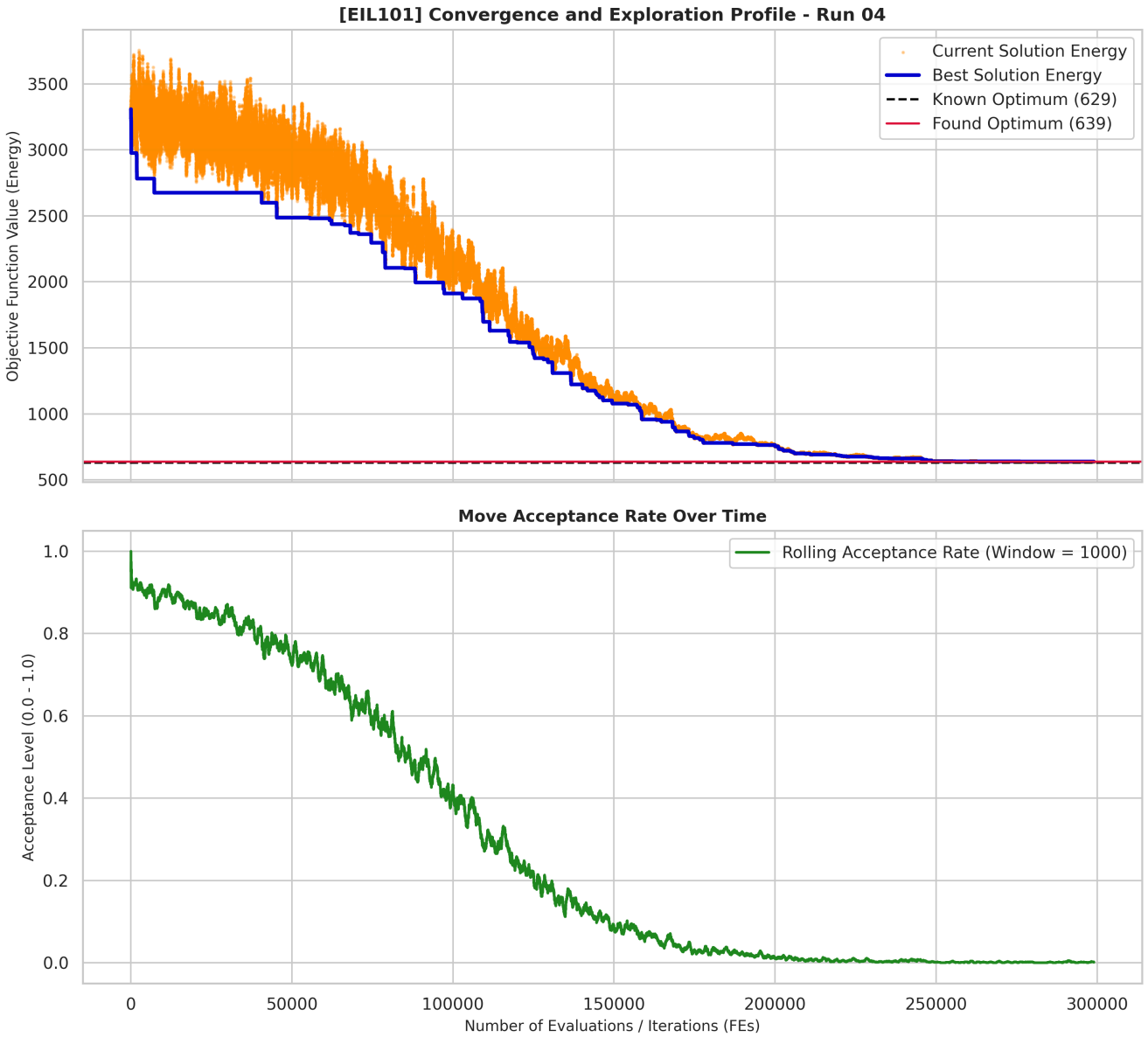


Figure 2: Optimization route cost and acceptance rate for SA on eil101 instance.

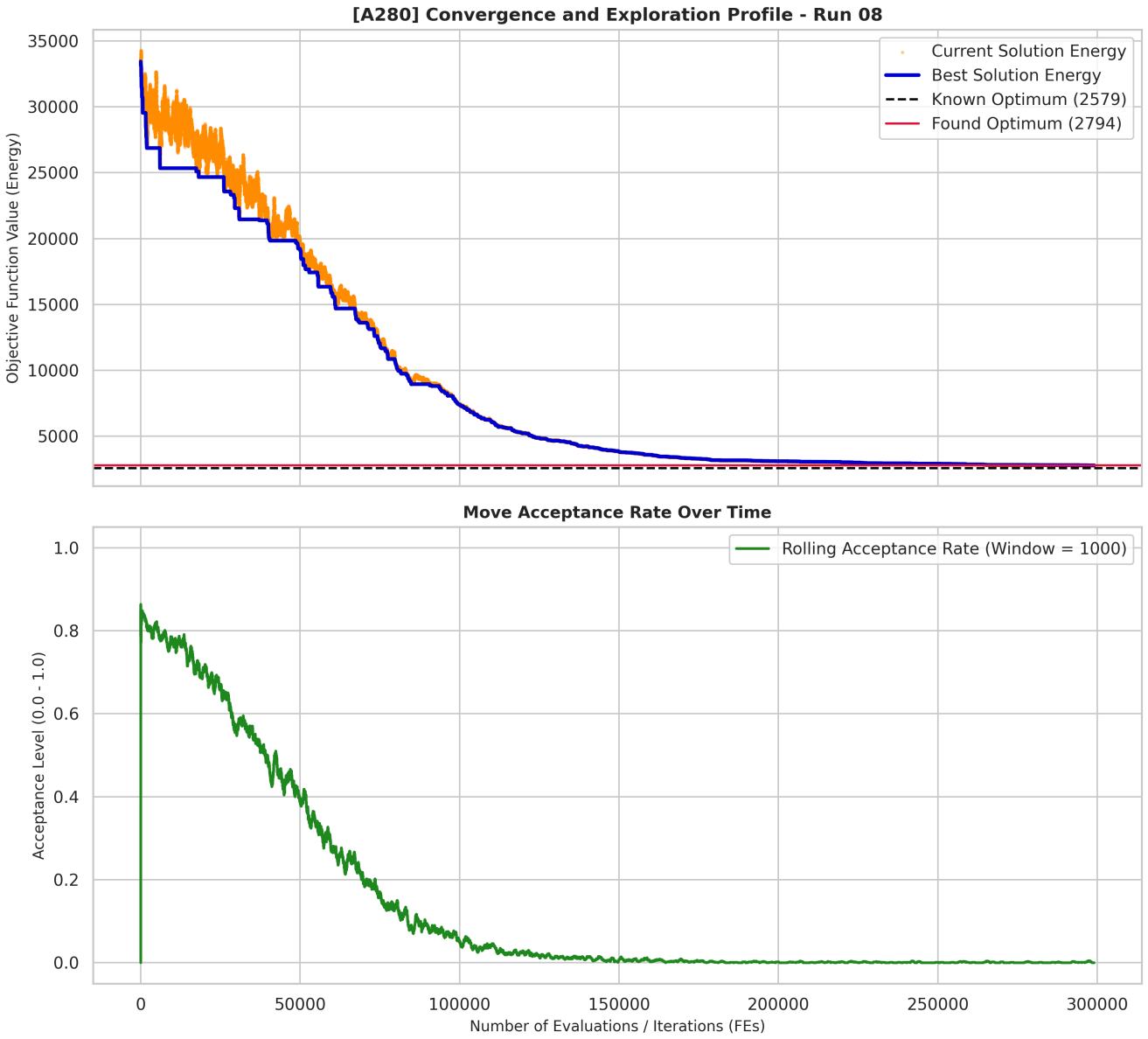


Figure 3: Optimization route cost and acceptance rate for SA on a280 instance.

6.3 Bees Algorithm

The final Bees Algorithm implementation finds valid TSP tours and consistently improves the quality of the initial population. The algorithm does not attempt to solve the problem exactly; instead, it combines constructive initialization, population-based selection and local 2-opt exploration. The constructive nearest-neighbour phase is important because it prevents the initial population from being dominated by very poor random tours. These constructed tours are still evaluated through the same objective function as every other solution, so they are included in the function evaluation budget.

The obtained results can be considered strong for a relatively simple population-based implementation. For `att48`, the best RPD was `0.78%`, which is very close to the known optimum. For `eil101`, the Bees Algorithm obtained the second best result among all three compared algorithms, with a best RPD of `2.86%` and an average RPD of `4.10%`. For the largest `a280` instance, the best result was `2719`, which gives a `5.43%` gap to the optimum `2579`. The average RPD on `a280` was `7.41%`, which is also better than the average result obtained by Simulated Annealing in the performed experiments.

The standard deviation of the RPD remains low for all tested instances: `1.15%` for `att48`, `0.77%` for `eil101` and `0.76%` for `a280`. This indicates that the final configuration is stable and does not rely on a

single lucky run. The execution time also scales favourably with the instance size. Even for `a280`, the average execution time was `3.39` seconds, which is much lower than the Ant System execution time and lower than the average time of Simulated Annealing in the comparative experiment.

The following charts present the results for the Bees Algorithm. The `eil101` instance is shown to keep the presentation consistent with the Ant System example above.

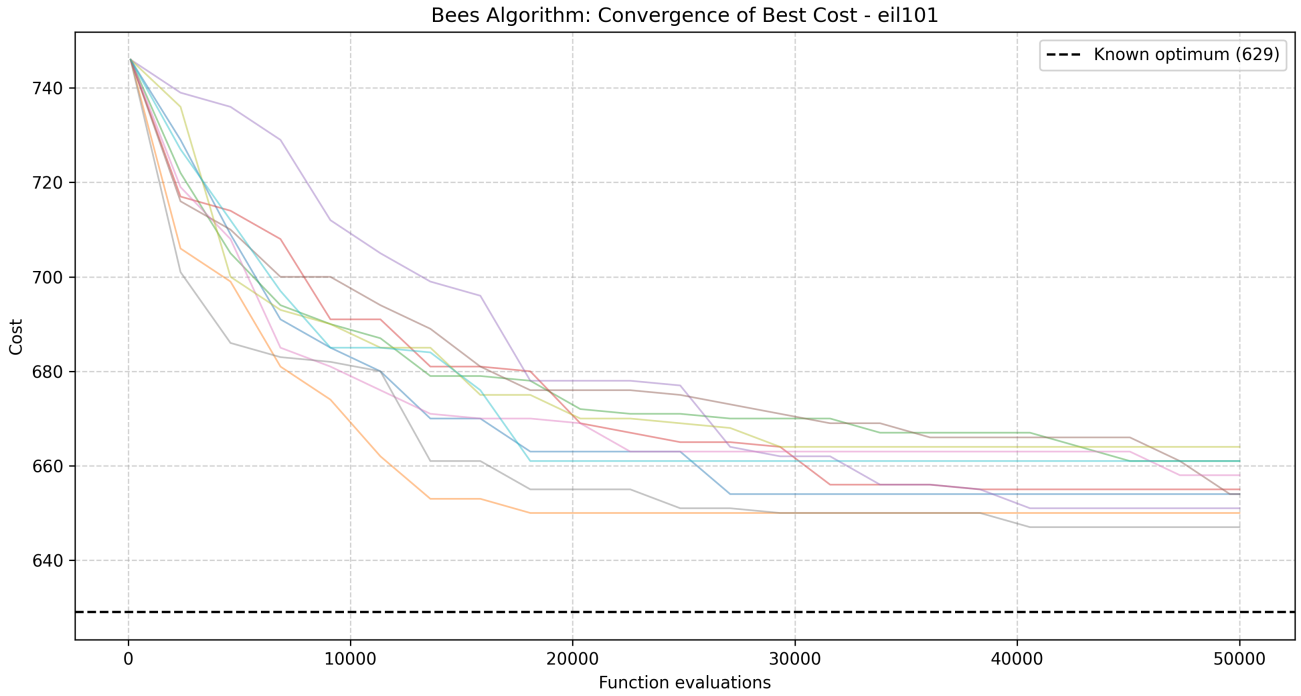


Figure 3: Best cost convergence for Bees Algorithm on eil101 across independent runs.

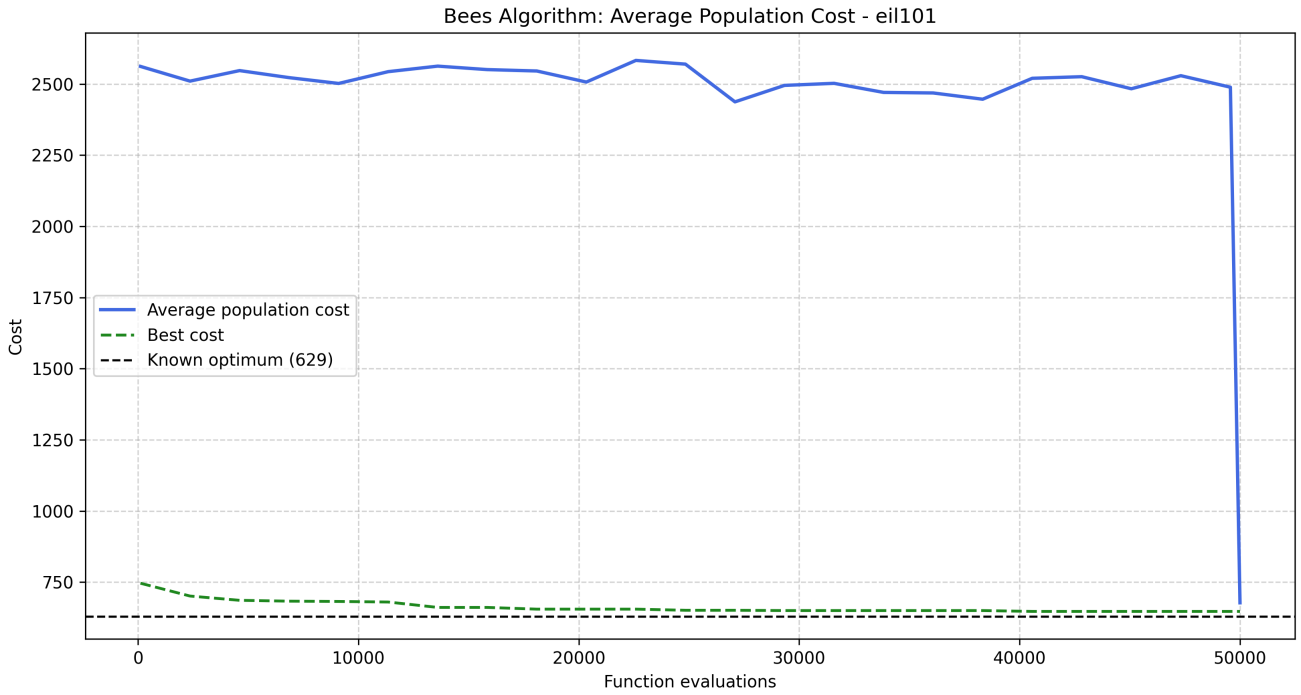


Figure 4: Average population cost and best cost for the best Bees Algorithm run on eil101.

The convergence plot confirms that the algorithm quickly moves from the constructive and random initial population to significantly shorter tours. The strongest improvements usually occur during the first part of the run, when recruited bees intensively explore the neighbourhoods of the best selected sites. Later

improvements become smaller, which is expected for a local-search-oriented metaheuristic operating close to a local optimum.

The average population cost plot shows the relation between exploitation and exploration. The best cost decreases monotonically because the algorithm stores the best solution found so far. The average population cost remains higher than the best cost, because the population still contains weaker scout solutions and non-elite sites. This is desirable: if the average population cost became almost identical to the best cost, it would indicate premature convergence and loss of exploration.

The standard deviation plot confirms that the population does not collapse into a single identical route. The standard deviation remains positive throughout the run, which means that the algorithm maintains diversity between candidate tours. This diversity is useful because it allows the algorithm to continue sampling different regions of the search space instead of repeatedly improving only one solution.

7. Algorithm Comparison

The three algorithms represent different search strategies. Ant System is a constructive population algorithm based on pheromone trails. Simulated Annealing is a single-solution trajectory method controlled by temperature and probabilistic acceptance of worse moves. Bees Algorithm is a population-based local search method that combines scout exploration with intensified search around the best selected sites. Because of these structural differences, the comparison should consider not only the final cost, but also stability, execution time, scalability and how the evaluation budget is spent.

In terms of solution quality, Simulated Annealing and Bees Algorithm clearly outperform the basic Ant System implementation on the tested instances. On `att48`, Simulated Annealing achieved the best result (`0.00%` RPD), while Bees Algorithm was very close (`0.78%` RPD). Ant System was weaker on this instance, with a best RPD of `3.17%`. On `eil101`, Simulated Annealing obtained the best result (`1.59%` RPD), followed by Bees Algorithm (`2.86%`) and Ant System (`6.68%`). On `a280`, Bees Algorithm again achieved the best result in this experiment (`5.43%` RPD), slightly ahead of Simulated Annealing (`8.34%`) and clearly ahead of Ant System (`14.93%`).

The average results lead to the similar general conclusion. Bees Algorithm had the best average RPD on `eil101` and `a280`, while Simulated Annealing had the best average RPD on `att48`. This suggests that the constructive nearest-neighbour initialization and intensive 2-opt exploitation used by Bees Algorithm become especially useful as the instance size increases. Ant System remained the weakest in terms of solution quality, especially for the largest instance, which is expected for a simple ant-cycle implementation without additional local improvement.

Stability is also important. Ant System had the lowest RPD standard deviation on `a280` (`0.65%`), but this stability came with consistently worse solutions. Bees Algorithm combined strong solution quality with low variance: its RPD standard deviation was below `1.2%` for all tested instances. Simulated Annealing was very strong, but its variance increased on the largest instance (`2.95%` RPD standard deviation), which indicates higher sensitivity to the randomness and cooling process.

Execution time shows the clearest practical difference. Ant System required the most time, especially on `a280`, where the average execution time was `55.51` seconds. This is caused by repeatedly constructing full tours for many ants and updating pheromone information. Simulated Annealing was much faster than Ant System, but it still used a larger number of function evaluations on average due to its long single-trajectory

search and stagnation criterion. Bees Algorithm was the fastest on the largest instance in the performed tests, with an average time of `2.77` seconds for `a280`, while still obtaining the best average RPD.

The evaluation budget also affects the interpretation of the comparison. The algorithms do not use identical stopping criteria: Ant System used a limit of 16 000 function evaluations and a stagnation condition, Simulated Annealing used a much higher maximum limit with an additional stagnation criterion, and Bees Algorithm used a fixed limit of 50 000 evaluations. Therefore, the comparison is not a proof that one algorithm is universally superior. It shows how the implemented variants behave under the chosen experimental setup. Under these conditions, Bees Algorithm offers the best compromise between solution quality, runtime and repeatability.

The plots support these conclusions. Ant System quickly reaches a near-final result and then changes only slightly, which suggests early stagnation. Simulated Annealing benefits from a longer trajectory and probabilistic acceptance, making it strong on small and medium instances. Bees Algorithm improves rapidly after initialization and keeps population diversity during the run, which helps it avoid complete premature convergence while still exploiting good routes intensively.

8. Final Conclusions

All three implemented metaheuristics are capable of producing valid TSP tours and improving them over time, but their practical behaviour differs significantly.

The Ant System implementation is stable and conceptually clear, but in the tested configuration it produces weaker solutions and has the highest execution time, especially for larger instances. Its main limitation is the lack of additional local route improvement after constructing tours.

Simulated Annealing is a strong baseline for TSP because the 2-opt neighbourhood and probabilistic acceptance mechanism allow it to escape poor local optima. It obtained the best result on the smallest instance and remained competitive on the larger ones, but its performance depends on the temperature schedule and the length of the trajectory.

The Bees Algorithm produced the best overall balance in the conducted experiments. The combination of nearest-neighbour initialization, selected-site neighbourhood search and scout exploration gave low RPD values, low variance and short execution times. Its strongest results were obtained on `a280`, where it achieved the best best-result RPD among the three algorithms.

The most important practical conclusion is that the neighbourhood operator and initialization strategy are crucial for TSP metaheuristics. Random permutations alone are too weak as starting points for larger instances, while 2-opt-based neighbourhood moves provide meaningful improvements by restructuring route segments. For future work, the Bees Algorithm could be further improved by automatic hyperparameter tuning, adaptive neighbourhood sizes, a stagnation-based stopping criterion, or a final deterministic 2-opt polishing phase applied to the best solution found.

---

9. Repository

- Github repository